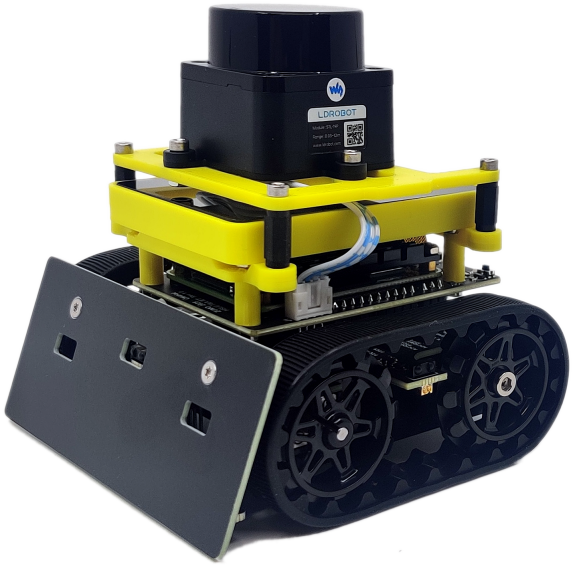
HSLU Hochschule Luzern

CSA
C# in Action

.NET auf Raspberry Pi / Zumo



23. März 2026

Christian Jost

Inhaltsverzeichnis

1 C# Deployment auf Linux **2**
1.1 Visual Studio Code . . . 3
1.2 Visual Studio Community Edition . . . 5

2 Remote Debugging von .NET Anwendungen **8**
2.1 Attach to Process (Visual Studio Community Edition) . . . 8

3 Remote Debugging von .NET Anwendungen (VSCode) **12**
3.1 Installation Debugger . . . 12
3.2 Launch Configuration . . . 12
3.3 Start Launch Configuration . . . 13
3.4 Auf den Debugger warten . . . 13

4 Zumo Peripherie **14**
4.1 Projekte erstellen . . . 14
4.2 NuGet Packages installieren . . . 15
4.3 Benutzer Led und Taster . . . 16
4.4 Lidar . . . 17
4.5 UART . . . 19
4.6 Antrieb . . . 19
4.7 RGB-LEDs . . . 20
4.8 ColorSensor . . . 20
4.9 Sound . . . 21

1 C# Deployment auf Linux

Weil üblicherweise der Host leistungsfähiger ist als das Zielsystem (Raspberry Pi), macht es Sinn, die Entwicklung auf dem Host zu machen und dann die Anwendung auf das Linux System mittels SSH zu übertragen. Hierzu stellen wir beim Benutzer auf dem Linux ein Verzeichnis `netcore`:

> **Hinweis:** Falls ein anderer Namen verwendet wird, dann muss der Name in den folgenden Schritten jeweils angepasst werden.

1. Auf dem Host überprüfen, ob die SSH Verbindung ohne Passworteingabe funktioniert: `ssh pi@pi-hslu`

   > **Info:** Je nach Netzwerk muss man 'pi-hslu' z.B. mit 'pi-home' ersetzen, siehe `config` Datei im `.ssh` Verzeichnis.

2. Im Home-Verzeichnis des Benutzers ein Unterverzeichnis `netcore` erstellen:

   ```
   pi@raspy:$ mkdir ~/netcore
   ```

3. Um die Speicherkarte oder den Flash-Speicher des Compute Moduls vor zu vielen Schreibzugriffen zu schützen, sollte die Entwicklung auf einem RAM-Drive erfolgen. Dies ist zudem auch schneller:

   - Die Datei /etf/fstab editieren:

     ```
     pi@raspy:$ sudo nano /etc/fstab
     ```

   - folgende Zeile unten anfügen, speichern und nano beenden:
     `tmpfs /home/pi/netcore tmpfs nodev,nosuid,size=100M 0 0`
   - Anschliessend das RAM-Drive einbinden:

     ```
     pi@raspy:$ sudo systemctl daemon-reload
     ```

     ```
     pi@raspy:$ sudo mount -a
     ```

   - Nun kann mit dem Befehl `df` geprüft werden, ob die Operation erfolgreich war:

     ```
     pi@zumo:~ $ df
     Filesystem     1K-blocks    Used Available Use% Mounted on
     udev             3730336       0   3730336   0% /dev
     tmpfs             799732    1204    798528   1% /run
     /dev/mmcblk0p2  29478824 3586928  24375184  13% /
     tmpfs            3998660       0   3998660   0% /dev/shm
     tmpfs               5120      12      5108   1% /run/lock
     tmpfs             102400       0    102400   0% /home/pi/netcore
     /dev/mmcblk0p1    522230   57514    464716  12% /boot/firmware
     tmpfs             799732       0    799732   0% /run/user/1000
     ```

     Abbildung 1: tmpfs für netcore Verzeichnis

4. Entweder ein existierendes .NET Projekt anpassen, oder im Visual Studio ein neues Projekt als Console App (.NET) erstellen.

   > **Hinweis:** Das Programm wird später auf dem Raspi ausgeführt, somit muss das Projekt vom Typ .NET sein (**nicht** etwa .NET Framework)! Auch muss die .NET Version mit der auf dem Raspberry Pi übereinstimmen.

1.1 Visual Studio Code

1. Erstellen Sie eine neue Konsolenanwendung oder verwenden Sie eine bestehende Anwendung.
2. Laden Sie die Zip-Datei `SecureUpload` herunter. Es gibt eine Version für Windows, Linux und MAC. Entpacken Sie die passende Datei in den Solution-Ordner, d.h. in das Verzeichnis, wo sich die **Solution** `.slnx` des Projektes befindet. Der Grund dafür ist, dass das nachfolgende Post-Build Script davon ausgeht, dass die Datei `SecureUpload` bzw. `SecureUpload.exe` im Verzeichnis der Solution Datei platziert ist.
3. Unter Linux/MAC muss evtl. die Ausführungsberechtigung gesetzt werden. Dazu im Terminal mittels `chmod ug+x SecureUpload` das Ausführungsbit für den Benutzer/Gruppe setzen.
4. Klicken Sie im SOLUTION-EXPLORER auf das Projekt. Dabei wird die Projektdatei mit der Endung `.csproj` geöffnet.
5. Fügen Sie folgende drei Zeilen in die Projektdatei ein:

```
<Target Name="PostBuild" AfterTargets="PostBuildEvent">
  <Exec Command="'$(SolutionDir)SecureUpload' '$(TargetDir)' 'zumo:netcore/$(ProjectName)'"/>
</Target>
```

Unter Linux/MAC kann alternativ die Variante mittels rsync verwendet werden:

```
<Target Name="PostBuild" AfterTargets="PostBuildEvent">
  <Exec Command="echo rsync -avz --delete $(TargetDir) zumo:netcore/$(ProjectName)"/>
</Target>
```

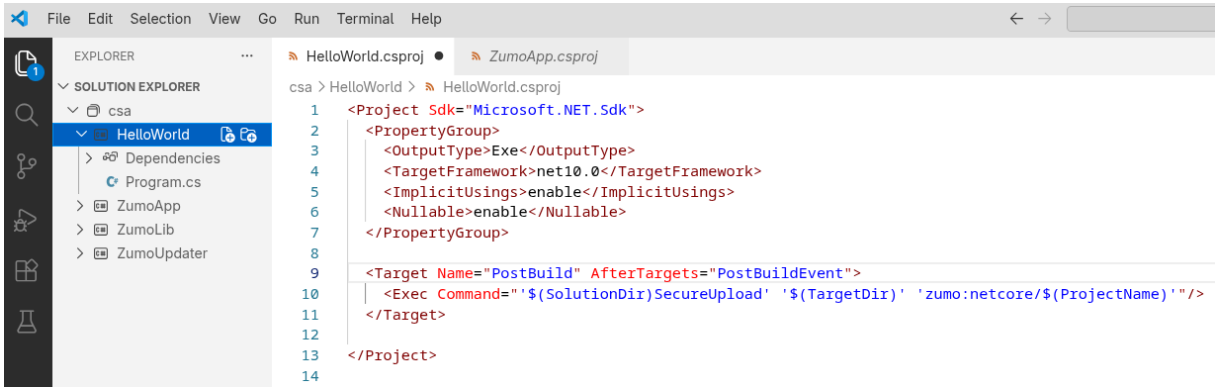


Abbildung 2: PostBuild-Event mit SecureUpload

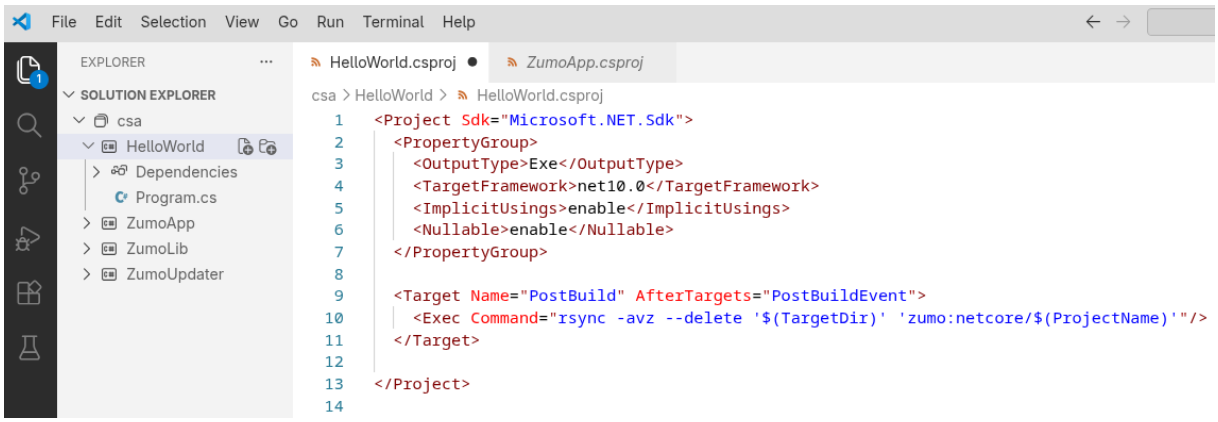


Abbildung 3: PostBuild-Event mit rsync

6. Wird das Programm nun (korrekt) übersetzt (*Build* oder *Rebuild*), so werden die Programmdateien direkt auf das das Linux System geladen.

7. Im Raspi kann man nun zum Projekt navigieren und das Programm ausführen. Der Start erfolgt nicht automatisch!

```
pi@raspy:$ dotnet <app>.dll
```

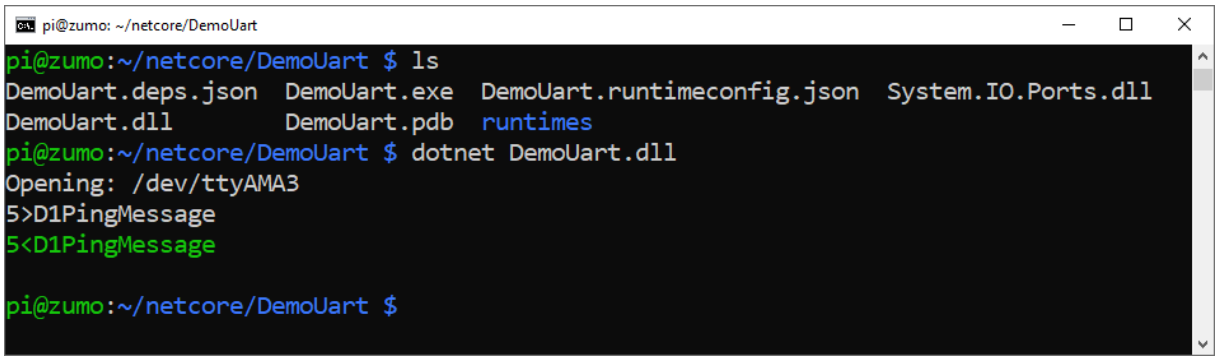


Abbildung 4: Hello Raspi World

1.2 Visual Studio Community Edition

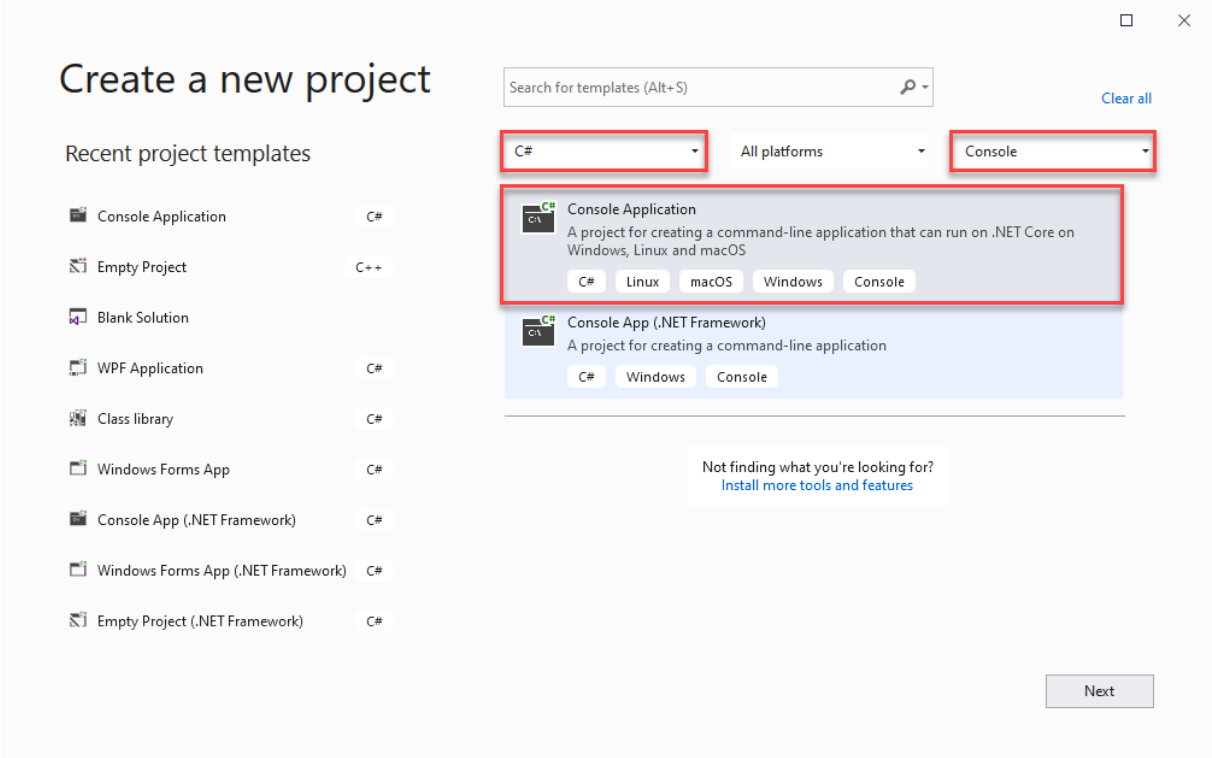


Abbildung 5: Erstellung neuer Konsolenanwendung

1. Der Quelltext sollte etwa so aussehen für ein 'Hello World':

```
using System;

namespace RaspyTest
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

2. Kopieren Sie die Datei `SecureUpload.exe` aus der Zip Datei in das Verzeichnis, wo sich die **Solution** `.sln` des Projektes befindet. Der Grund dafür ist, dass das nachfolgende Post-Build Script davon ausgeht, dass die Datei `SecureUpload.exe` im Verzeichnis der Solution Datei platziert ist.

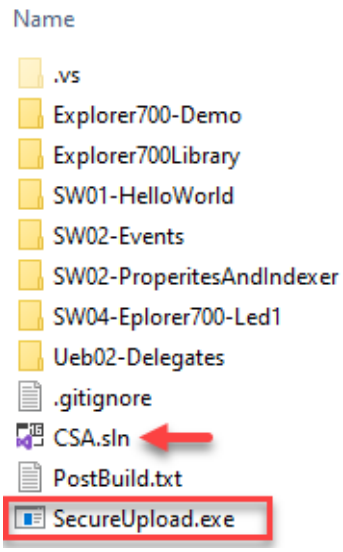


Abbildung 6: SecureUpload.exe

3. Öffnen Sie die Projekt-Eigenschaften (Properties) und füge die folgenden Zeilen für den Post-Build Event hinzu (Abbildung 7) Die Zeilen findet man auch im `SecureUpload.zip`. Die Kommandos verwenden die Namen/Verzeichnisse mit doppelten Anführungszeichen, um auch mit einem Pfad umzugehen, der Leerzeichen beinhaltet.

> **Hinweis:** Verwende den aktuellen Hostnamen des Raspberry Pi, z.B. pi-hslu oder pi-home, je nach Netzwerk.

```
cd "$(TargetDir)"
"$(SolutionDir)\SecureUpload.exe" . pi-home:netcore/$(ProjectName)
```

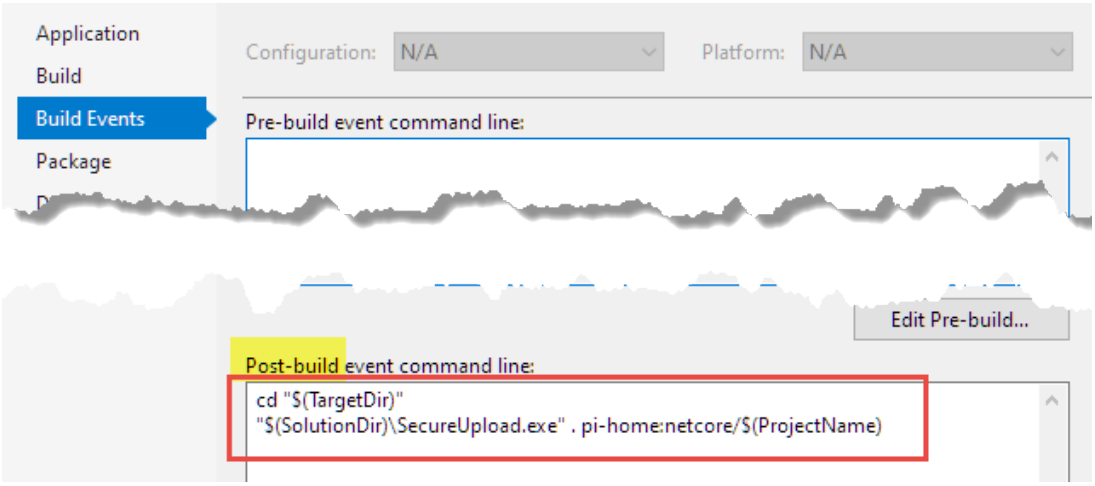


Abbildung 7: Post Build Event

> **Info:** Sollte der Post-Build Step im Visual nicht funktionieren, kann man entweder WinSCP verwenden oder die Dateien via Konsole kopieren.

Mit dem Post-Build Event wird nach einem Build in das Projektverzeichnis gewechselt, und mittels `SecureUpload` die Dateien und Verzeichnis auf das Raspberry in das Verzeichnis `netcore` kopiert.

4. Wird das Programm nun (korrekt) übersetzt (*Build* oder *Rebuild*), so werden die Programmdateien direkt auf das das Linux System geladen.

5. Im Raspi kann man nun zum Projekt navigieren und das Programm ausführen. Der Start erfolgt nicht automatisch!

```
pi@raspy:$ dotnet <app>.dll
```

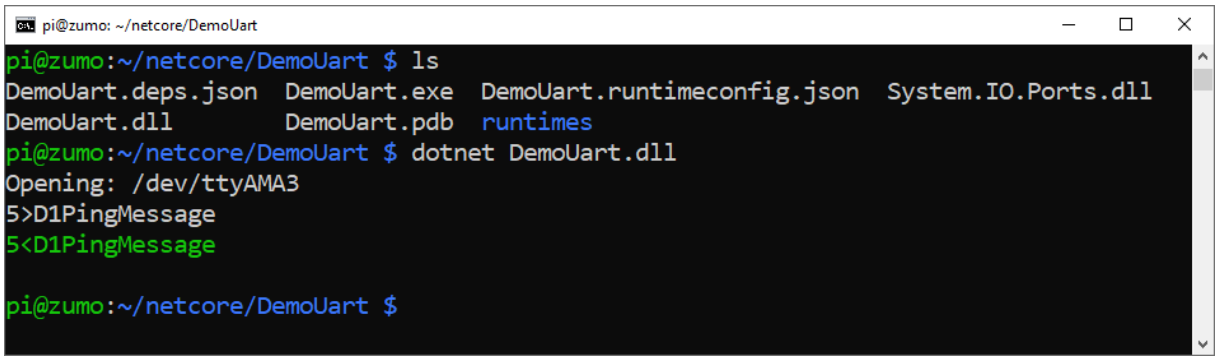


Abbildung 8: Hello Raspi World

2 Remote Debugging von .NET Anwendungen

2.1 Attach to Process (Visual Studio Community Edition)

Als nächstes wollen wir uns vom Host aus mit einem laufenden Programm unter Linux verbinden und es debuggen. Hierzu erweitern wir unser Programm, damit wir genügend Zeit haben, um uns darauf zu verbinden und einen Breakpoint zu setzen:

```
using System;
using System.Threading;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello World!");
        for (int i = 0; i < 30; i++)
        {
            Console.Write(".");
            Thread.Sleep(1000);
        }
    }
}
```

Im Visual Studio konfigurieren wir nun den Debugger, damit er sich auf den Prozess auf dem Raspy verbindet. Hierzu verwenden wir das Menu `Debug` → `Attach to Process...` (Abbildung 9).

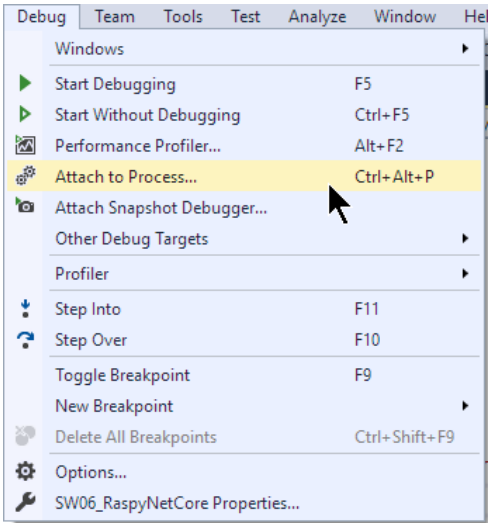


Abbildung 9: Attach to Process

Im Dialog macht man folgendes:

1. Connection type: **SSH**
2. Connection target: **Hostnamen des Raspy**
3. **Refresh** Button drücken

Im Dialog eingeben (Abbildung 10):

1. Benutzernamen
2. **Private Key** Methode

3. Private Key Dateiname

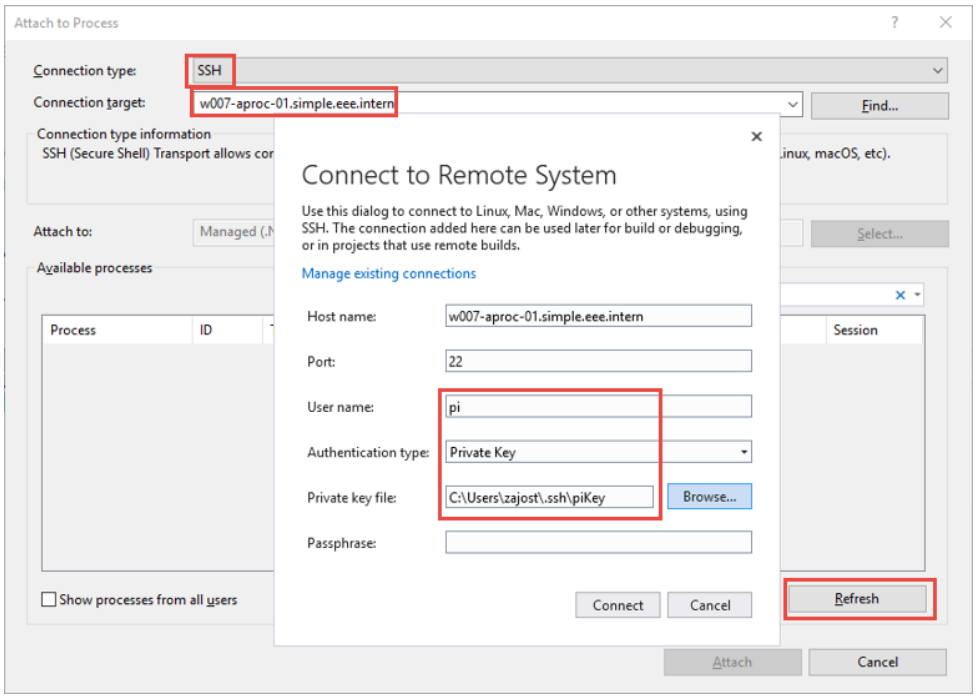


Abbildung 10: Attach to Process Einstellungen

Mittels `Connect` Button kann man dann auf das Target verbinden und es zeigt die Liste der Prozesse an (Abbildung 11)

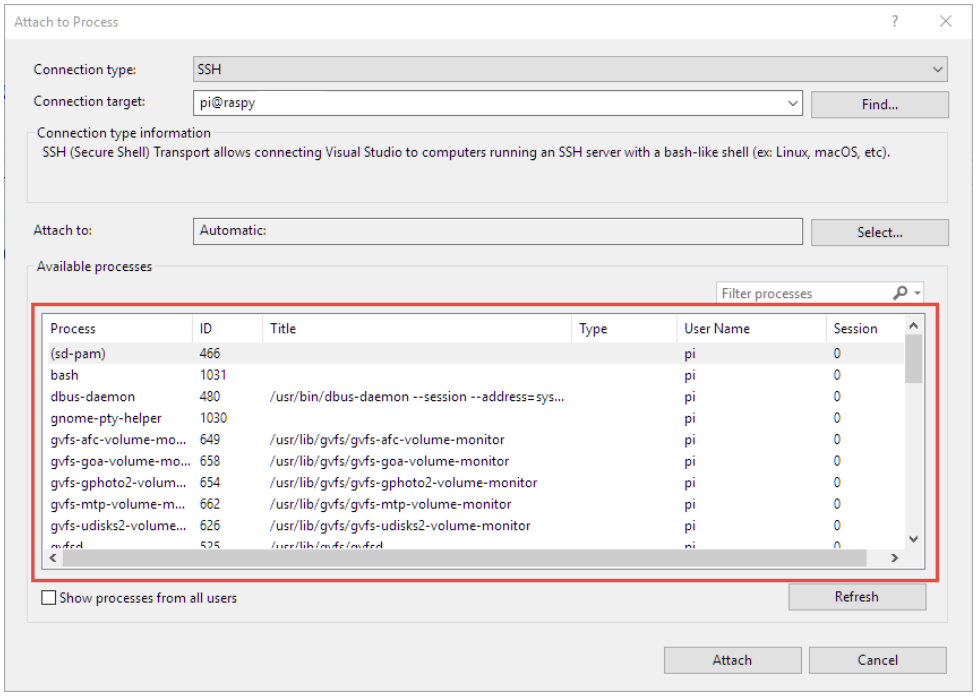


Abbildung 11: Liste der Prozesse auf Linux

Nun können wir den Prozess manuell auswählen und uns dann darauf verbinden (Abbildung 12):

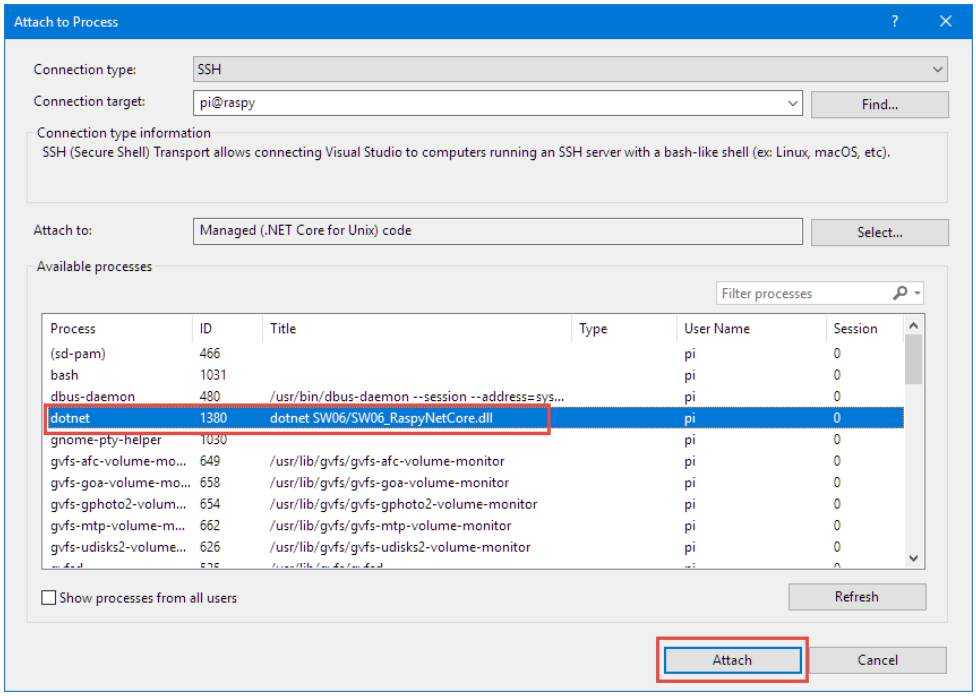


Abbildung 12: dotnet Prozess in Liste

Da die Suche etwas mühsam sein kann, könne wir die Liste der Prozesse filtern. Mit **Select...** können wir sagen, dass wir nur an den **.NET Core for Unix** Prozessen interessiert sind (Abbildung 13).

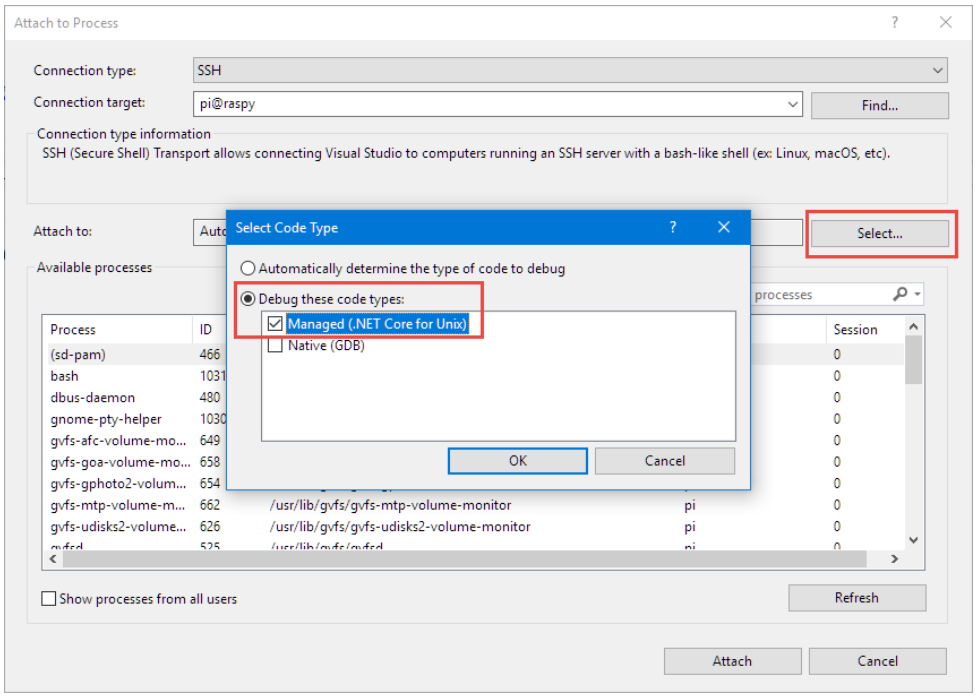


Abbildung 13: Managed .NET Core für Unix Prozess Selektion

Selbstverständlich muss das Programm auf dem Raspy dazu laufen, deshalb haben wir eine Warteschlaufe integriert. Wenn man die Anwendung auf dem Raspy startet, hat man nun Zeit, eine Verbindung mit dem VVisual Studio herzustellen (Abbildung 14):

```
pi@raspyerichstyger:~/netcore $ dotnet SW06/SW06_RaspyNetCore.dll
Hello World!
...
```

Abbildung 14: Wartende Anwendung

Mit den Bedienelementen im Visual Studio können wir den Linux Prozess anhalten bzw. stoppen (Abbildung 15):

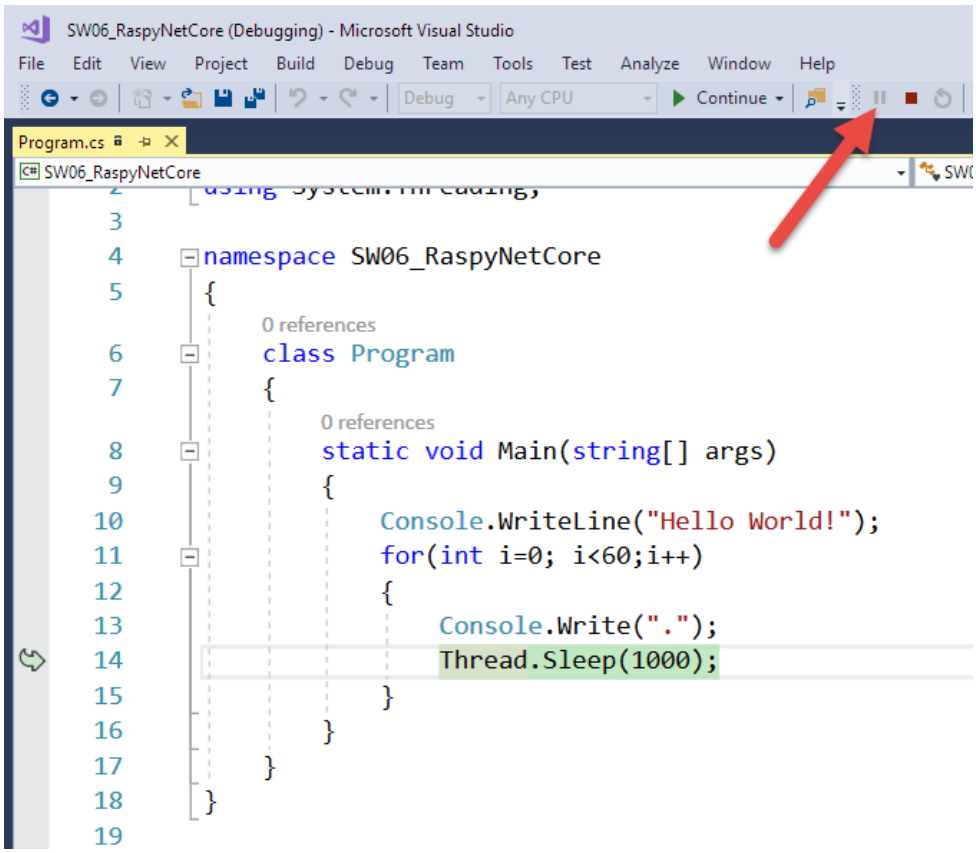


Abbildung 15: Managed .NET Core für Unix Prozess Selektion

Hiermit können wir nun vom Host PC aus ein .NET Projekt auf dem Linux System (Raspberry Pi) debuggen.

Achtung: Je nach Version kommt es beim ersten Verbindungsaufbau des Debuggers zu einer Fehlermeldung, dass ein Verzeichnis mit dem Namen .vs-debugger nicht gefunden wurde. In dem Fall muss im Home Verzeichnis mit dem Befehl `mkdir ~/.vs-debugger` dieses Verzeichnis noch erstellt werden.

3 Remote Debugging von .NET Anwendungen (VSCode)

3.1 Installation Debugger

Mit Visual Studio Code ist es möglich, .NET Anwendungen remote zu debuggen. Dazu muss auf dem Zielsystem (Raspberry Pi) der VSDBG Server installiert werden. Quelle: Attaching to remote process.

Dazu in einer Raspberry Pi Console folgende Anweisung ausführen:

```
wget https://aka.ms/getvsdbgsh -O - 2>/dev/null | /bin/sh /dev/stdin -v latest -l ~/vsdbg
```

3.2 Launch Configuration

Im VSCode muss eine passende Launch Configuration erstellt werden. Mit dem Dateiexplorer oder im VSCode in der Workspace Ansicht (nicht Solution Explorer) die Datei launch.json wie in der Abbildung 16 gezeigt erstellen und editieren.

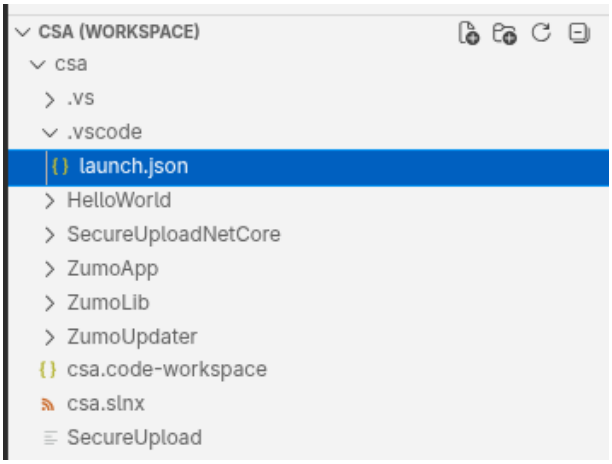


Abbildung 16: Launch Configuration File

Der Inhalt der Datei soll wie folgt aussehen (auch verfügbar über ILIAS):

```
{
    // Use IntelliSense to learn about possible attributes.
    // Hover to view descriptions of existing attributes.
    // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
    "version": "0.2.0",
    "configurations": [
        {
            "name": ".NET SSH Attach",
            "type": "coreclr",
            "request": "attach",
            "processId": "${command:pickRemoteProcess}",
            "pipeTransport": {
                "pipeProgram": "ssh",
                "pipeArgs": [
                    "-T",
                    "pi-hslu"
                ],
                "debuggerPath": "~/vsdbg/vsdbg",
                "pipeCwd": "${workspaceRoot}",
                "quoteArgs": true
            },
        },
    ]
}
```

Falls nötig den Eintrag der SSH-Config übernehmen, hier: pi-hslu

3.3 Start Launch Configuration

Um eine Anwendung remote zu debuggen, diese auf dem Zielsystem starten und anschliessend im VSCode die erstellte Launch Configuration auswählen:

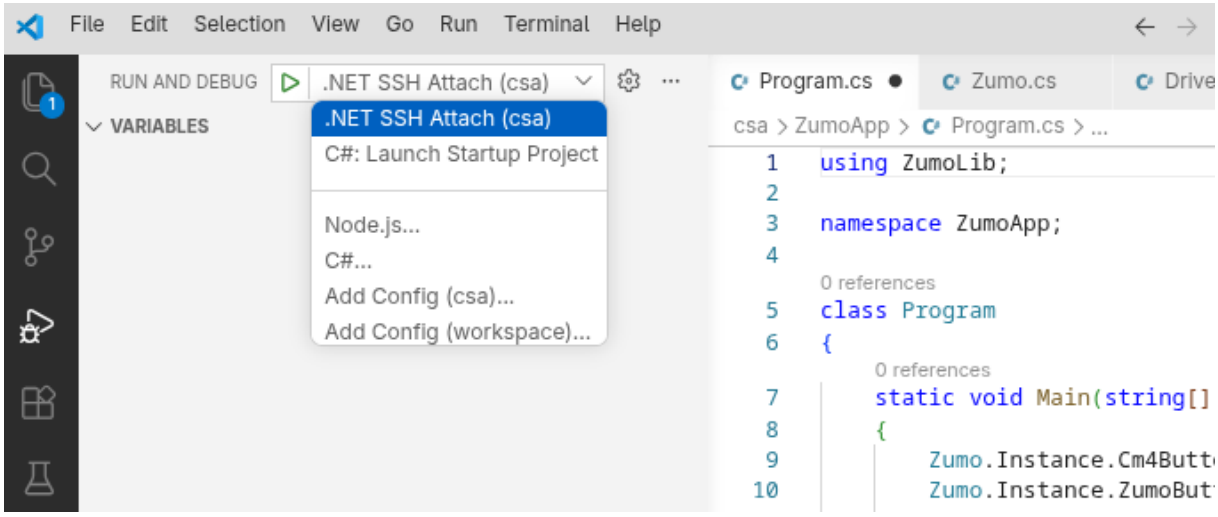


Abbildung 17: Select Launch Configuration

Anschliessend kann das Debugging mittels grünem Play-Knopf oder der Funktionstaste F5 gestartet werden. Dabei muss auf dem Zielsystem der dotnet Prozess ausgewählt werden.

3.4 Auf den Debugger warten

Das Programm muss bereits laufen, bevor man sich mit dem Debugger verbinden kann. Um ab der ersten Anweisung debuggen zu können, muss das Programm zu Beginn warten, bis der Debugger verbunden ist. Folgendes Code-Snippet kann dabei helfen:

```
public static bool WaitForDebugger()
{
    string[] args = Environment.GetCommandLineArgs();
    if (args.Length > 1 && args[1] == "--debug")
    {
        ConsoleColor color = Console.ForegroundColor;
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine("Waiting for Debugger or press <Enter> to continue");
        Console.ForegroundColor = color;
        while (true) {
            if (Console.KeyAvailable) {
                if (Console.ReadKey().Key == ConsoleKey.Enter) {
                    break;
                }
            }
            if (System.Diagnostics.Debugger.IsAttached) {
                return true;
            }
            Thread.Sleep(100);
        }
    }
    return false;
}
```

4 Zumo Peripherie

Um die Peripherie des Zumo Roboters in C# einzubinden, erstellen wir zwei Projekte im Visual Studio: ZumoLib und ZumoApp. In der Klassenbibliothek ZumoLib werden universelle Klassen für den Zugriff auf die Hardware erstellt. Die Anwendung mit ihrer Logik wir in der Applikation ZumoApp Implementiert.

4.1 Projekte erstellen

Kopieren Sie die Projekte in Ihren Solution Ordner. Anschliessen können Sie mit Rechtsklick auf die Solution die bestehenden Projekte mittels Add->Existing Project... hinzufügen. Falls Sie später selber ein neues Projekt erstellen, müssen Sie die Projektreferenz zur Bibliothek manuell hinzufügen, indem Sie mittels Rechtsklick auf Ihr Projekt den Menüpunkt ”Add Project Reference...” auswählen (siehe Abbildung 18 bzw. Abbildung 19).

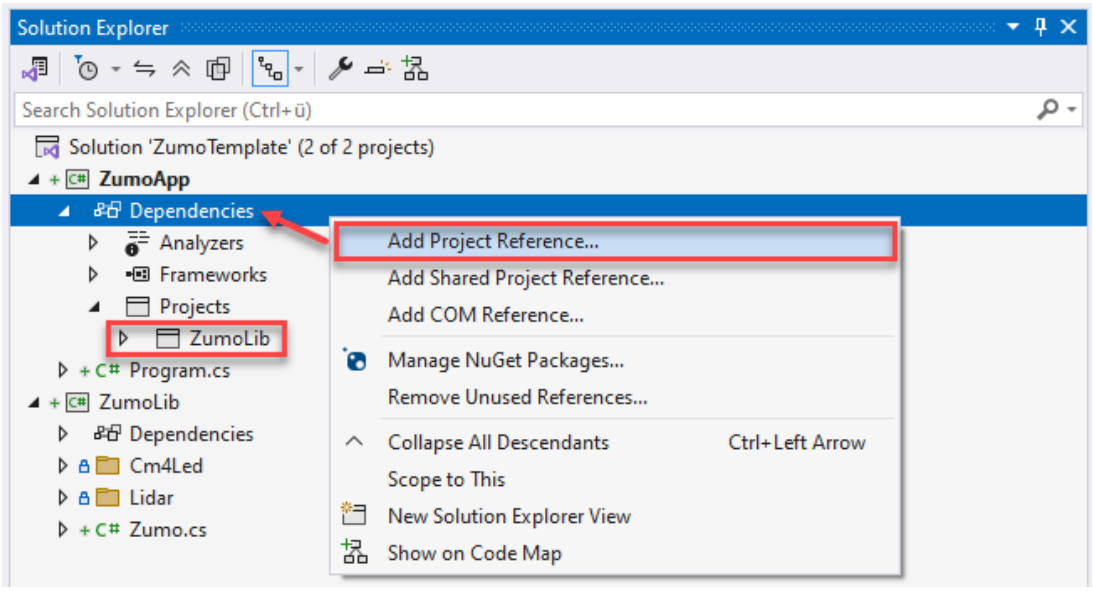


Abbildung 18: Projektreferenz hinzufügen (Visual Studio)

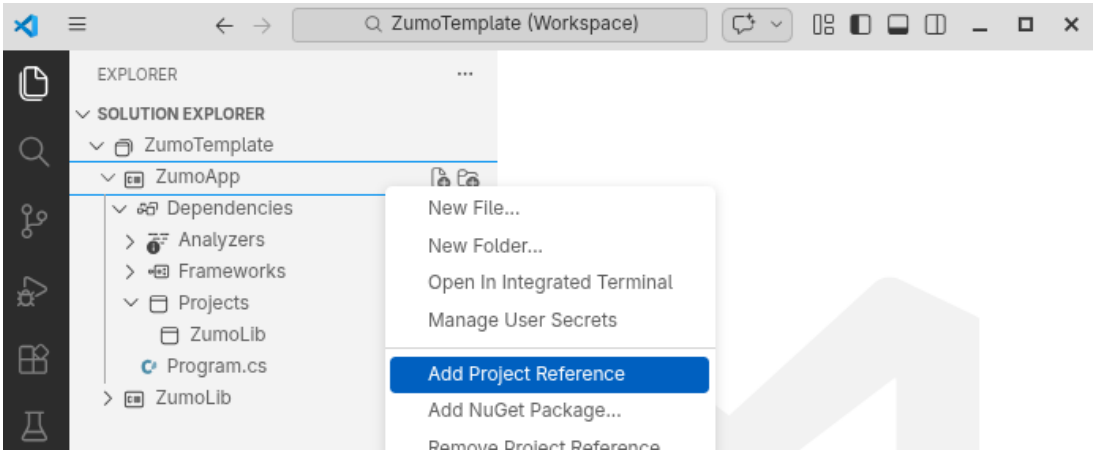


Abbildung 19: Projektreferenz hinzufügen (Visual Studio Code)

4.2 NuGet Packages installieren

Im bestehenden Projekt von ILIAS sollten die zusätzlich benötigen Bibliotheken für den Hardwarezugriff bereits installiert sein. Sie können dies wie folgt prüfen: Öffnen Sie den NuGet Packager Manager mittels Rechtsklick auf das Projekt und installieren Sie die nachfolgenden Packages, falls sie noch nicht installiert sind:

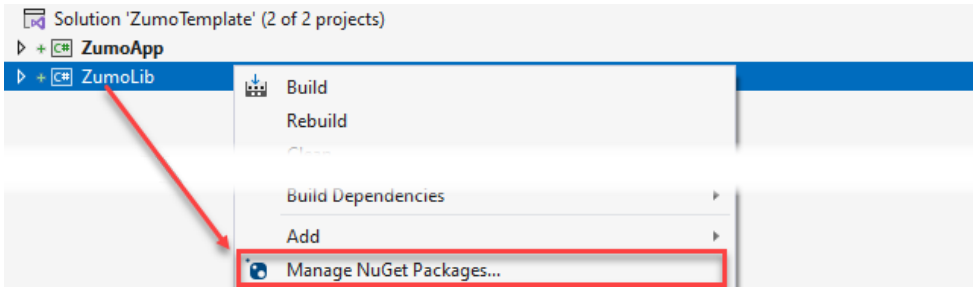


Abbildung 20: NuGet PackageManager (Visual Studio)

Folgende Packages müssen vorhanden sein oder installiert werden:

- System.Device.Gpio
- System.IO.Ports

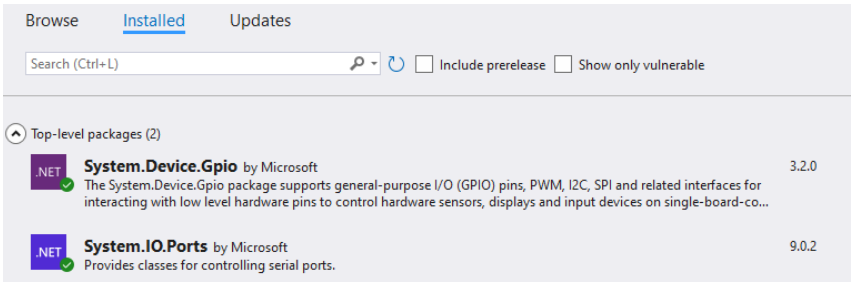


Abbildung 21: NuGet Packages (Visual Studio)

Im VSCode ist die Verwaltung der NuGets nicht ganz so bequem wie im Visual Studio:

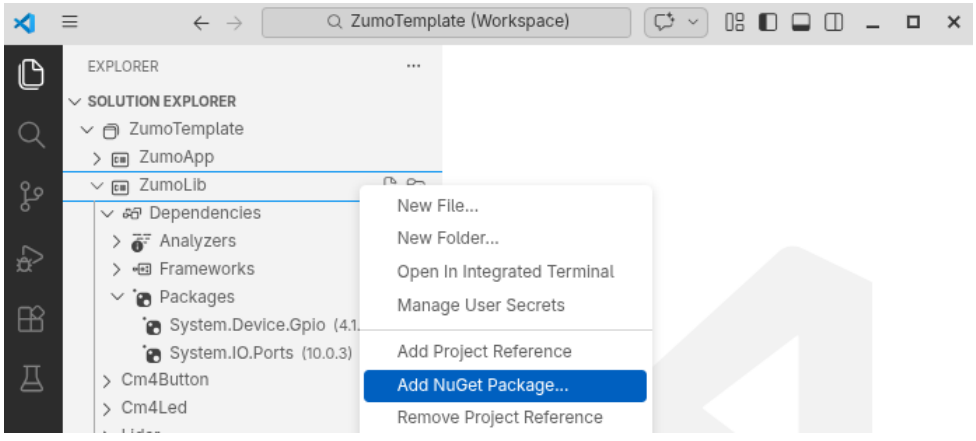


Abbildung 22: NuGet PackageManager (Visual Studio Code)

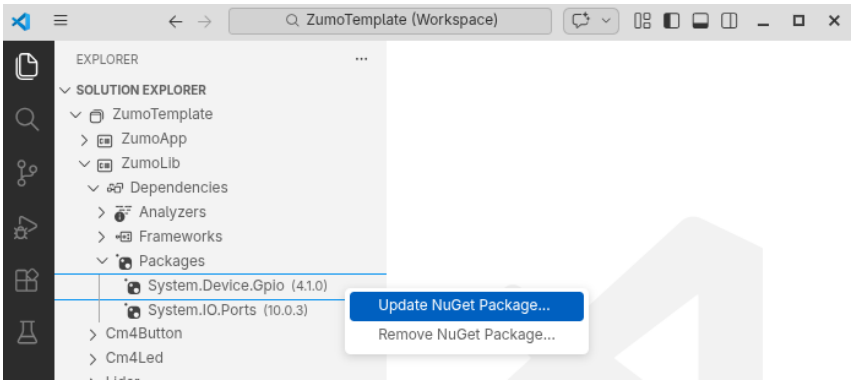


Abbildung 23: NuGet Packages (Visual Studio Code)

4.3 Benutzer Led und Taster

Am linken Rand des CM4-Boards gibt es eine Led sowie einen Taster, die mittels CM4 Board angesteuert werden können. Die Led ist mit dem GPIO Pin 18 verbunden. Der Button ist am Pin 27 angeschlossen und liefert im Ruhezustand einen High-Pegel.

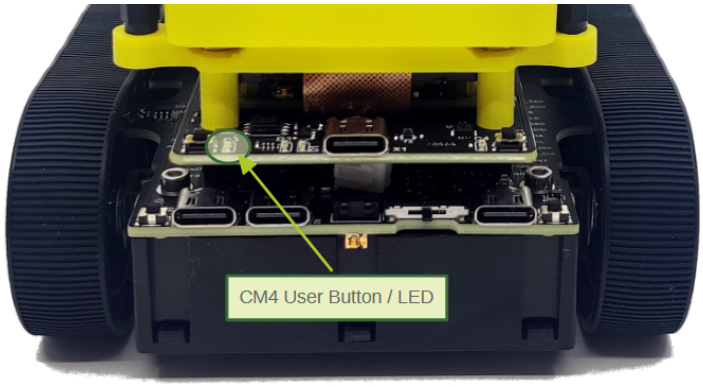


Abbildung 24: CM4 Benutzer Led

Mit Hilfe der Klasse `System.Device.Gpio.GpioController` wird der GPIO Pin im Konstruktor der CM4Led Klasse geöffnet und als Ausgang definiert.

Anschliessend kann mittels Read und Write Methoden der Pin auf High bzw. Low und damit die Led ein-/ausgeschaltet werden. Diese Funktionalität soll im Property `Enabled` der Klasse Cm4Led passieren.

Auf die selbe Art kann der der Taster abgefragt werden. Sobald sich sein Zustand ändert, soll das Event `ButtonChanged` ausgelöst werden. Der Button soll alle 50 ms abgefragt werden.

4.4 Lidar

Lidar (Light Detection and Ranging) ist eine Technologie, die mithilfe von Lichtstrahlen Entfernungen misst und dabei eine zwei- oder dreidimensionale Abbildung der Umgebung erstellt. Dies funktioniert wie folgt:

- Ein Lidar-System sendet eine sehr schnelle Serie von Lichtimpulsen aus. Diese Impulse bestehen in der Regel aus Infrarotlicht, das für das menschliche Auge unsichtbar ist.
- Diese Lichtimpulse treffen auf Objekte in der Umgebung und werden von diesen reflektiert. Der Lidar-Sensor misst die Zeit, die der Lichtimpuls benötigt, um vom Sensor zum Objekt und zurückzukehren. Diese Zeit wird in Nanosekunden gemessen.
- Mithilfe der Lichtgeschwindigkeit und der gemessenen Zeit berechnet der Lidar die Entfernung zu den Objekten.
- Indem der Lidar-Sensor viele Impulse in verschiedene Richtungen sendet, erstellt er eine Sammlung von Punkten (”Punktwolke”). Dazu wird der Sensor mittels Motor mehrmals pro Sekunde um die eigene Achse gedreht.



Abbildung 25: LD19 Lidar Modul

Um das Lidar Modul auf dem Zumo Roboter steuern zu können, muss in der Datei /boot/firmware/config.txt der Pin 12 als PWM Pin konfiguriert werden. Da es eine Überlappung mit der UART5 gibt, muss diese deaktiviert werden (falls sie aktiviert ist):

```
enable_uart=1
dtoverlay=uart2
dtoverlay=uart4
dtoverlay=uart3
#dtoverlay=uart5

dtoverlay=gpio-shutdown,gpio_pin=3,active_low=1
dtoverlay=gpio-poweroff,gpio_pin=26

dtparam=i2c_vc=on
dtoverlay=i2c-rtc,pcf85063a,i2c_csi_dsi

dtoverlay=pwm,pin=12,func=4
```

Abbildung 26: Zumo Lidar Konfiguration

Anschliessend Linux neu starten.

Implementieren Sie in der Klasse Lidar die Thread Methode Run. In der Endlosschleife soll die Daten das Lidar Sensors gelesen und ausgewertet werden. Eine mögliche Lösung sieht wie folgt aus:

- In einer Endlosschleife Byte für Byte prüfen, ob es sich um das erste Header-Byte handelt.
- Falls das erste Byte korrekt ist, prüfen ob das nächste Byte auch korrekt ist. Falls nicht, wieder an den Anfang und warten bis das erste Byte gültig ist.
- Falls beide Header-Bytes korrekt sind, dann warten bis 45 Bytes empfangen wurden und die CRC Prüfsumme über alle 47 Bytes bilden. Falls diese nicht 0 ergibt => wieder an den Anfang.
- Falls die Prüfsumme korrekt ist, in einer inneren Endlosschleife jeweils ein Datenpaket (47 Bytes) lesen, auswerten und die Daten in das 360° Point Array speichern.
- Im Fehlerfall die innere Endlosschleife verlassen und erneut auf die Header-Bytes warten, d.h. wieder an den Anfang gehen.
- Die Daten können wie folgt ausgewertet werden:
  - Byte 2 und Byte 3 liefern die Geschwindigkeit
  - Byte 4 und Byte 5 liefern den Winkel (Anfang des Fensters)
  - Byte 42 und Byte 43 liefern den Winkel (Ende des Fensters)
  - Byte 44 und Byte 45 liefern eine Zeitinformation (wird nicht verwendet)
  - Byte 46 die Prüfsumme
  - Byte 6 bis und mit Byte 41 liefern die 12 x 3 Bytes an Daten.

4.5 UART

Befehle für Antrieb, LEDs, Farbsensor etc. können via UART an den Zumo gesendet werden. Dabei muss jeder Befehl mit einem LineFeed abgeschlossen werden. In den Beispielen unten steht `5>` für einen Set-Befehl und `5?` für eine Abfrage. Nach dem Set/Get Zeichen `5>24` bzw. `5?24` wählt das Modul an welches die Anfrage gesendet wird, in diesem Beispiel 0x24 (Antrieb). Die erste Hex-Ziffer nach dem Modul ist der Unterbefehl von diesem Modul. Signed 16-Bit-Werte werden im Zweierkomplement übertragen, also zum Beispiel `005A` für `+90` und `FFA6` für `−90`.

Verbindungscheck:
`5>D1Ping`
Antwort: `5<D1Ping`

4.6 Antrieb

Das Antriebsmodul besitzt die Dispatcher-Adresse `0x24`, die Quadraturgeber die Adresse `0x22`.

- **Fahre Strecke vorwärts/rückwärts** (`SSSS`: Distanz in mm, `VVVV`: Geschwindigkeit in mm/s, `AAAA`: Beschleunigung in mm/s^2):
  `5>242SSSSVVVVAAAA`
  Positive Distanz fährt vorwärts, negative Distanz rückwärts. Während der Fahrt kann mit `5?242` die verbleibende Distanz abgefragt werden.

- **Drehe an Ort** (`SSSS`: Winkel in Grad, `VVVV`: Geschwindigkeit in mm/s, `AAAA`: Beschleunigung in mm/s^2):
  `5>24ASSSSVVVVAAAA`
  Positive Winkel drehen im Uhrzeigersinn, negative gegen den Uhrzeigersinn. Beispiel: `5>24A005A03E803E8` dreht den Roboter um `+90` ° mit `1000` mm/s und `1000` mm/s^2.

- **Fahre mit konstanter Geschwindigkeit** (`LLLL`: linkes Rad, `RRRR`: rechtes Rad in mm/s):
  `5>241LLLLRRRR`
  Beide Werte sind signiert. Positive Werte bedeuten Vorwärtsfahrt des jeweiligen Rades, negative Werte Rückwärtsfahrt.

- **Kurvenfahrt mit Radius** (`SSSS`: Winkel in Grad, `RRRR`: Radius in mm, `VVVV`: Geschwindigkeit in mm/s, `AAAA`: Beschleunigung in mm/s^2):
  `5>249SSSSRRRRVVVVAAAA`
  Damit lässt sich ein definierter Kreisbogen fahren. Positive Winkel bedeuten Rechtskurve, negative Winkel Linkskurve. Radius ist das Zentrum des Roboters.

- **Kalibrierfaktor für Drehen an Ort setzen** (`NNNN`: Faktor):
  `5>24BNNNN`
  Dies ist ein Hilfsbefehl zum Feinabgleich des Winkels bei Drehen an Ort. Standardmässig ist der Skalierungsfaktor auf $\frac{130}{100}$ (Faktor $1{,}3$) gesetzt. Zur Kalibrierung diesen Faktor auf 100 setzen (`5>24B0064`), einen Winkel (z. B. $180°$ oder $360°$) drehen und den tatsächlich erreichten Winkel messen. Den zu berechnen

$$\text{Faktor} = \frac{\text{Sollwinkel}}{\text{Istwinkel}} \cdot 100$$

- **Aktuelle Sollgeschwindigkeit abfragen**:
  `5?241`
  Antwort-Payload: `LLLLRRRR`

- **Verbleibende Distanz einer aktiven Streckenfahrt abfragen**:
  `5?242`
  Antwort-Payload: `SSSS`
- **Quadraturgeschwindigkeit links/rechts abfragen**:
  `5?220`
  Antwort-Payload: `0LLLLRRRR`. Die Werte sind die aktuell gemessenen Radgeschwindigkeiten.
- **Quadraturdistanz links/rechts abfragen**:
  `5?221`
  Antwort-Payload: `1LLLLRRRR`. Die Distanzen sind bereits in Millimetern umgerechnet.
- **Quadraturdistanz zurücksetzen**:
  `5>220`
  Setzt beide Distanzzähler auf null.
- **Quadratur-Distanzfaktor setzen**:
  `5>221NNNN`
  Setzt den Distanz-Kalibrierfaktors für die Encoder-Auswertung. Abfrage mit `5?222`. Zur Kalibrierung 1000mm fahren und die tatsächlich gefahrene Strecke messen und diese als Faktor setzen. Beispiel bei 950mm effektiv gemessen: `5>22103B6`

4.7 RGB-LEDs

Die beiden oberen RGB-LEDs besitzen die Dispatcher-Adresse `0x12`, die acht unteren RGB-LEDs an der Front die Adresse `0x11`. Das Format für die Ansteuerung ist identisch:

- **Farbe setzen** (`LL`: LED-Bitmaske, `RR`: Rot, `GG`: Grün, `BB`: Blau):
  `5>11LLRRGGBB` für die unteren LEDs bzw.
  `5>12LLRRGGBB` für die oberen LEDs.
  Die Bitmaske wählt aus, welche LEDs gesetzt werden. Bei den unteren LEDs bedeutet beispielsweise `01` die erste LED, `02` die zweite LED und `FF` alle acht LEDs gleichzeitig.
- **Beispiel alle unteren LEDs rot**:
  `5>11FFFF0000`
- **Beispiel alle unteren LEDs blau**:
  `5>11FF0000FF`
- **LEDs ausschalten**:
  `5>110F` bzw. `5>120F`
- **Farbe einer LED abfragen**:
  `5?11LL` bzw. `5?12LL`
  Antwort-Payload: `LLRRGGBB`

4.8 ColorSensor

Der Farbsensor besitzt die Dispatcher-Adresse `0x31`. Die Set- und Get-Unterbefehle erlauben sowohl die direkte Farbabfrage in RGB oder als Hue Winkel.

- **Farbton abfragen**:
  `5?310`
  Antwort-Payload: `HHHH`. Zurückgegeben wird der berechnete Farbton in Grad. Bei zu dunklem oder zu wenig gesättigtem Signal wird ein spezieller Ungültig-Wert zurückgeliefert.

- **Schwarz/Weiss-Kalibrierung ausführen**:
  `5>316XX`
  `XX=00`: Referenz schwarzer Untergrund aufnehmen
  `XX=01`: Referenz weisser Untergrund aufnehmen und Kalibrierung anwenden

4.9 Sound

Der Sound-Treiber besitzt die Dispatcher-Adresse `0x50`. Aktuell sind zwei Set-Befehle implementiert: ein einzelner Piepton und das Abspielen einer vordefinierten Melodie.

- **Piepsen** (`FFFF`: Frequenz in Hz, `TTTT`: Dauer in ms):
  `5>500FFFFTT`
  Beispiel für `440 Hz` während `1000 ms`:
  `5>50001B803E8`

- **Musik abspielen**:
  `5>501N` mit einem Song-Index im Low-Nibble des letzten Hex-Bytes.
  - `0>5010`: Knight Rider
  - `0>5011`: Star Wars
  - `0>5012`: Super Mario
  - `0>5013`: Wasted
  - `0>5014`: Harry Potter
  - `0>5015`: Lord of the Rings
  - `0>5016`: Indiana Jones
  - `0>5017`: James Bond 007
  - `0>5018`: Pacman